

Module 06

NVIDIA Ecosystem

GPU, NeMo, dan RAG yang dipercepat



Act 1

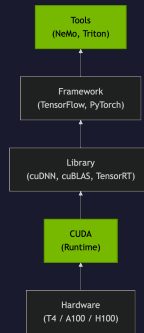
GPU, CUDA & Optimasi

Fondasi hardware & software NVIDIA

Stack NVIDIA: Dari Chip ke Tools

Lapisan-lapisan yang membuat AI berjalan cepat

- ▶ **Hardware** = GPU (T4/A100/H100)
- ▶ **CUDA** = jembatan agar GPU bisa untuk komputasi umum
- ▶ **Library** = cuDNN, cuBLAS, TensorRT (rutin matematika siap pakai)
- ▶ **Framework** + **Tools** = TF/PyTorch + NeMo/Triton (tempat kita menulis kode)



Tiap lapisan berdiri di atas lapisan bawahnya: kita menulis di Framework, GPU yang bekerja keras.

Kenapa GPU? (Recap Module 02)

Ribuan core paralel untuk operasi matriks

- ▶ **CPU**: sedikit core, sangat pintar, kerja berurutan
- ▶ **GPU**: ribuan core sederhana, kerja **paralel** (serentak)
- ▶ **Deep learning** = tumpukan **matrix operations** (perkalian matriks)
- ▶ Perkalian matriks dipecah jadi ribuan operasi kecil → pas untuk GPU

Lihat **Module 02** untuk dasar arsitektur GPU. Catatan: notebook ini hanya mengukur waktu di GPU — perbandingan vs CPU adalah pengetahuan umum, bukan benchmark di sini.

Benchmark Model: microsoft/phi-2

Mengukur kecepatan inference di GPU

- ▶ Memuat **microsoft/phi-2** (~2.7B parameter)
- ▶ `torch_dtype=float16 (FP16)` → hemat memori GPU
- ▶ `device_map="auto"` → taruh di GPU otomatis
- ▶ **Warmup** 1x, lalu `generate()` 5x, dirata-rata

```
1 model = AutoModelForCausalLM.  
    from_pretrained(  
2     "microsoft/phi-2",  
3     torch_dtype=torch.float16,    # FP16  
4     device_map="auto")           # ke GPU
```

Pola load model NVIDIA: **FP16** + `device_map="auto"`. Benchmark = rata-rata waktu `generate` di GPU (tanpa baseline CPU).

Half Precision: FP16 vs FP32

Angka 16-bit untuk hemat memori dan lebih cepat

- ▶ **Mixed precision** (campur FP32+FP16): konsep umum pelatihan model
- ▶ **Tensor Cores**: unit GPU NVIDIA super cepat untuk FP16
- ▶ Notebook ini memuat phi-2 dalam **FP16 penuh** (`torch.float16`), *bukan* mixed precision (tanpa AMP/autocast)

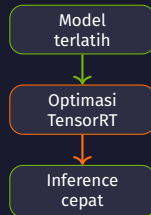
Tipe	Bit	Sifat
FP32	32-bit	Presisi penuh, boros memori
FP16	16-bit	Hemat memori ~50%, cepat di Tensor Cores

Yang dipakai notebook: half precision (FP16 penuh). Mixed precision (AMP) dibahas di Module 02, tidak didemokan di sini.

TensorRT: Optimasi Inference

Membuat model berjalan jauh lebih cepat saat dipakai

- ▶ **TensorRT**: alat NVIDIA mempercepat **inference** (saat dipakai)
- ▶ **Layer fusion + kernel auto-tuning**
- ▶ **Precision calibration**: turun ke FP16/INT8, akurasi terjaga
- ▶ **TF-TRT**: tipikal $2-5\times$ lebih cepat (angka NVIDIA)



Di Module 06 TensorRT hanya **di-install** & dibahas konseptual — tidak ada demo (notebook memakai PyTorch).

Act 2

NVIDIA NeMo

Toolkit untuk Conversational AI

Apa itu NeMo?

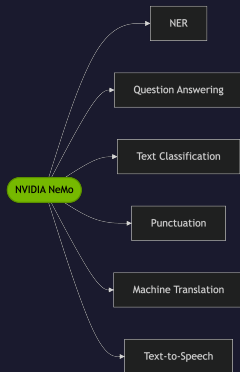
Toolkit conversational AI dari NVIDIA

- ▶ **NeMo** (Neural Modules): toolkit **open-source** NVIDIA untuk **conversational AI** (AI percakapan)
- ▶ Cakupan: pemahaman teks, terjemahan, sampai sintesis suara
- ▶ Kaya **pre-trained model** (siap pakai) — tak perlu melatih dari nol
- ▶ Modular: komponen dirangkai bebas seperti balok lego
- ▶ Skalabel karena dioptimalkan untuk **GPU** NVIDIA

Inti NeMo: unduh model AI canggih, panggil method-nya, langsung pakai — tanpa training dari awal.

Peta Tugas NeMo

6 task conversational AI yang didemokan



Satu toolkit, enam task — semuanya cukup dengan **pre-trained model** siap pakai.

NER & Question Answering

Memahami dan menjawab dari teks

- ▶ **NER:** TokenClassificationModel + ner_en_bert (berbasis **BERT**)
- ▶ add_predictions() menandai entitas: "NVIDIA" → ORG, "New York" → LOC
- ▶ **QA:** QAModel + qa_squadv1.1_bertbase
- ▶ Model prediksi posisi start & end jawaban di konteks

```
1 model = TokenClassificationModel \  
2     .from_pretrained("ner_en_bert")  
3 results = model.add_predictions(  
    texts)
```

NER menandai SIAPA/DI MANA; QA menunjuk potongan teks (start-end) yang menjawab pertanyaan.

Text Classification & Punctuation

Mengelompokkan dan merapikan teks

- ▶ **Text Classification:**

TextClassificationModel → kategori (mis. sentimen)

- ▶ `classify()` memberi label per kalimat

- ▶ **Punctuation:**

PunctuationCapitalizationModel +
`punctuation_en_bert`

- ▶ Contoh: "nvidia is a company" → "Nvidia is a company."

```
1 text = "nvidia is a company"  
2 model.  
   add_punctuation_capitalization(  
3   [text])[0]
```

Klasifikasi memberi label; punctuation mengubah teks polos jadi rapi (berguna untuk hasil transkripsi).

Machine Translation & TTS

Lintas bahasa dan teks ke suara

- ▶ **MT:** MTEncDecModel + nmt_en_zh_transformer24x6 (Inggris → Mandarin)
- ▶ `translate()` terima daftar kalimat
- ▶ **TTS:** pipeline 2 tahap berurutan
- ▶ Tahap 1 FastPitch: teks → **spectrogram**
- ▶ Tahap 2 HiFiGAN (**vocoder**): spectrogram → audio

```
1 parsed = spec_gen.parse(text)
2 spec = spec_gen.generate_spectrogram(
3     tokens=parsed)
4 audio = vocoder \
5     .convert_spectrogram_to_audio(spec=spec)
```

TTS = 2 tahap: teks jadi spectrogram (FastPitch), lalu spectrogram jadi audio (HiFiGAN).

Catatan Penting

Sebelum menjalankan demo NeMo

- Demo ini **berat di GPU** — jangan dijalankan semua sekaligus
- ✓ Sifatnya **konseptual**: fokus memahami ALUR tiap task
- Notebook meng-import model **ASR** (speech recognition) & MegatronGPTModel...
- ...tetapi keduanya **TIDAK didemonstrasikan** — import $\neq$ dipakai
- ✓ Lihat **Module 02** (dasar GPU) & **Module 05** (dasar RAG)

Baca kode dengan teliti: meng-**import** model (ASR & Megatron) tidak sama dengan men-**demokan**-nya.

Act 3

NVIDIA-Accelerated RAG

RAG dipercepat GPU

RAG di atas Stack NVIDIA

Menjalankan RAG dengan akselerasi GPU

- ▶ **RAG (Retrieval-Augmented Generation)** = pencarian dokumen + **LLM** agar jawaban berbasis fakta (dasar di **Module 05**)
- ▶ Fokus di sini: sudut **tooling NVIDIA** — **embedding** & LLM dipindah ke **GPU** (`.to('cuda')`)
- ▶ Install **faiss-gpu** bersama `nemo_toolkit`, `transformers`



Arsitektur NvidiaRAGSystem

Empat komponen dalam satu pipeline

- ▶ **Embedding:** SentenceTransformer (all-MiniLM-L6-v2) di GPU
- ▶ **Index:** FAISS (IndexFlatL2)
- ▶ **Generator:** LLM **gpt2** di GPU
- ▶ **Doc:** **cudf** (RAPIDS) bungkus dokumen
→ pandas

```
1 self.embedding_model = SentenceTransformer(  
2     'all-MiniLM-L6-v2').to(self.device)  
3 self.llm = AutoModelForCausalLM \  
4     .from_pretrained("gpt2").to(self.device)  
5 df = cudf.DataFrame(  
6     {'text': documents}).to_pandas()
```

cudf hanya membungkus dokumen lalu langsung `.to_pandas()` (chunking pakai Python biasa).

Module o5 vs Module o6: Tooling NVIDIA

RAG yang sama, tooling berbeda

Aspek	Module o5 (RAG dasar)	Module o6 (NVIDIA)
Penempatan model	<code>device_map="auto", float32</code>	GPU eksplisit (<code>.to(cuda)</code>)
Embedding	all-MiniLM-L6-v2	all-MiniLM-L6-v2
LLM	TinyLlama (float32)	gpt2
FAISS	faiss-cpu	faiss-gpu (install)
Doc handling	Python biasa	cudf (RAPIDS) → pandas

Pipeline RAG tetap sama; yang ditukar adalah **tooling**-nya.

Alur Demo & Catatan

`demonstrate_rag()`: retrieve → augment → generate

- ▶ **Retrieve:** `retrieve_relevant_chunks()` cari potongan termirip via FAISS ($k=3$)
- ▶ **Augment:** gabung query + konteks jadi satu **prompt**
- ▶ **Generate:** `generate_response()` pakai gpt2 menulis jawaban

Catatan akurasi: MegatronGPTModel hanya di-*import*; LLM yang dipakai = **gpt2**.
faiss-gpu di-install, tapi IndexFlatL2 sebenarnya dibangun di **CPU**.

Ringkasan & Akhir Course

Ekosistem NVIDIA menutup perjalanan kita

Module 06:

- ▶ **GPU/CUDA:** fondasi akselerasi
- ▶ **NeMo:** toolkit conversational AI siap pakai
- ▶ **RAG** dipercepat: faiss-gpu, model di GPU

Perjalanan course:

- ✓ 01 Machine Learning
- ✓ 02 Deep Learning
- ✓ 03 NLP 04 LLM
- ✓ 05 RAG 06 NVIDIA

Dari **machine learning** klasik hingga **LLM & RAG** yang dipercepat GPU — pondasi lengkap untuk membangun AI modern.

Terima Kasih!

Selamat — Anda telah menyelesaikan seluruh course!

NVIDIA Ecosystem · Modul 06
Navasena Gen-ML Course